

Modificar Cliente – parte 1



En esta lección, hay varios cambios que hacer para poder mostrar el resultado visto en el video de editar un cliente.

Los archivos de `editar-cliente.component.ts`, `editar-cliente.component.html` y `cliente.service.ts` cambiaron, varios cambios debido a que estamos usando la última versión de Angular, TypeScript y Firestore, por lo tanto las 3 tecnologías sufrieron cambios. Recuerden que les adjuntamos el proyecto resuelto actualizado y funcionando con los cambios necesarios en estos archivos.

Código Actualizado de `editar-cliente.component.html`

```
<header id="main-header" class="py-2 bg-info text-white">
  <div class="container">
    <div class="row">
      <div class="col-md-6">
        <h1><i class="fas fa-user"></i> Editar Cliente</h1>
      </div>
    </div>
  </div>
</header>

<form #clienteForm="ngForm" (ngSubmit)="guardar(clienteForm)">

  <!-- Aquí se muestran los mensajes de alerta -->
```

```

<jjwins-alert-messages></jjwins-alert-messages>

<!-- Botones -->
<section id="actions" class="py-4 mb-4 bg-light">
  <div class="container">
    <div class="row">
      <div class="col-md-3">
        <a routerLink="/" class="btn btn-primary btn-block">
          <i class="fas fa-arrow-left"></i> Regresar al Inicio
        </a>
      </div>
      <div class="col-md-3">
        <button type="submit" class="btn btn-success btn-block">
          <i class="fas fa-check"></i> Guardar Cliente
        </button>
      </div>
      <div class="col-md-3">
        <a href="#" (click)="eliminar()" class="btn btn-danger btn-block">
          <i class="fas fa-trash"></i> Eliminar Cliente
        </a>
      </div>
    </div>
  </div>
</section>

<section id="details">
  <div class="container">
    <div class="row">
      <div class="col">
        <div class="card">
          <div class="card-header">
            <h4>Editar Cliente</h4>
          </div>
          <div class="card-body">
            <div class="mb-3">
              <label for="nombre" class="form-label">Nombre</label>
              <input
                type="text"
                name="nombre"
                class="form-control"
                #nombre="ngModel"
                [(ngModel)]="cliente.nombre"
                [ngClass]="{ 'is-invalid': nombre.errors && nombre.touched }"
                required
                minlength="2"

```

```

        />
        <div *ngIf="nombre.errors && nombre.touched" class="text-danger">
            <div *ngIf="nombre.errors['required']">Nombre es
requerido.</div>

            <div *ngIf="nombre.errors['minlength']">
                Debe contener al menos 2 caracteres.
            </div>
        </div>
    </div>
    <div class="mb-3">
        <label for="apellido" class="form-label">Apellido</label>
        <input
            type="text"
            name="apellido"
            class="form-control"
            [(ngModel)]="cliente.apellido"
            #apellido="ngModel"
            required
            minlength="2"
            [ngClass]="{ 'is-invalid': apellido.errors && apellido.touched
}"
        />
        <div *ngIf="apellido.errors && apellido.touched" class="text-
danger">

            <div *ngIf="apellido.errors['required']">Apellido es
requerido.</div>

            <div *ngIf="apellido.errors['minlength']">
                Debe contener al menos 2 caracteres.
            </div>
        </div>
    </div>
    <div class="mb-3">
        <label for="email" class="form-label">Email</label>
        <input
            type="email"
            name="email"
            class="form-control"
            [(ngModel)]="cliente.email"
            #email="ngModel"
            required
            [ngClass]="{ 'is-invalid': email.errors && email.touched }"
            pattern="^\\w+([\\.-]?\\w+)*@\\w+([\\.-]?\\w+)*\\.\\w{2,3}+$"
        />
        <div *ngIf="email.errors && email.touched" class="text-danger">
            <div *ngIf="email.errors['required']">Email es requerido.</div>

```

```
<div *ngIf="email.errors['pattern']">Email no es válido.</div>
</div>
</div>
<div class="mb-3">
  <label for="saldo" class="form-label">Saldo</label>
  <input
    type="number"
    name="saldo"
    class="form-control"
    [(ngModel)]="cliente.saldo"
    #saldo="ngModel"
    required
    [ngClass]="{ 'is-invalid': saldo.errors && saldo.touched }"
  />
  <div *ngIf="saldo.errors && saldo.touched" class="text-danger">
    <div *ngIf="saldo.errors['required']">Saldo requerido.</div>
  </div>
</div>
</div>
</div>
</div>
</div>
</section>
</form>
```

Código actualizado de editar-cliente.component.ts:

```
import { Component, OnInit } from '@angular/core';
import { Cliente } from '../modelo/cliente.modelo';
import { ClienteServicio } from '../servicios/cliente.service';
import { AlertMessagesService } from 'jjwins-angular-alert-messages';
import { ActivatedRoute, Router } from '@angular/router';
import { NgForm } from '@angular/forms';

@Component({
  selector: 'app-editar-cliente',
  templateUrl: './editar-cliente.component.html',
  styleUrls: ['./editar-cliente.component.css'],
})
export class EditarClienteComponent implements OnInit {
  clientes: Cliente[] = [];
  cliente: Cliente = {
```

```
nombre: '',
apellido: '',
email: '',
saldo: undefined,
};

id: string;

constructor(
  private clientesServicio: ClienteServicio,
  private alertMessageService: AlertMessagesService,
  private router: Router,
  private route: ActivatedRoute
) {}

ngOnInit() {
  this.id = this.route.snapshot.params['id'];
  this.clientesServicio
    .getClientes(this.id)
    .subscribe((cliente: Cliente | null) => {
      if (cliente) {
        this.cliente = cliente;
      } else {
        // Manejar el caso en el que el cliente es null
        console.warn('Cliente no encontrado');
      }
    });
}

eliminar() {
  throw new Error('Method not implemented.');
}

guardar(ngForm: NgForm) {
  throw new Error('Method not implemented.');
}
}
```

Código actualizado de cliente.service.ts:

```
import { Injectable } from "@angular/core";
import { Cliente } from "../modelo/cliente.modelo";
import { Observable } from "rxjs";
```

```
import { AngularFire, AngularFireCollection, AngularFireDocument } from
'@angular/fire/compat/firestore';
import { map } from 'rxjs/operators';

@Injectable()
export class ClienteServicio {

  clientesColeccion: AngularFireCollection<Cliente>;
  clientes: Observable<Cliente[]>;

  constructor(private firestore: AngularFire) {
    // Inicializa la colección con la referencia y la consulta
    this.clientesColeccion = this.firestore.collection<Cliente>('clientes', ref =>
ref.orderBy('nombre', 'asc'));

    // Obtén los datos de la colección
    this.clientes = this.clientesColeccion.valueChanges({ idField: 'id' });
  }

  getClientes(): Observable<Cliente[]> {
    return this.clientes;
  }

  agregarCliente(cliente: Cliente) {
    this.clientesColeccion.add(cliente);
  }

  getCliente(id: string): Observable<Cliente | null> {
    return this.firestore.doc<Cliente>(`clientes/${id}`).valueChanges({ idField: 'id' }).pipe(
      map(cliente => cliente ? cliente : null)
    );
  }
}
```

Comparación y Explicación de los Cambios:

Cambio:

- Se actualizó el paquete de `@angular/fire/firestore` a `@angular/fire/compat/firestore`. La versión `compat` es utilizada para mantener la compatibilidad con las versiones anteriores de AngularFire, permitiendo a los desarrolladores migrar gradualmente a la API más moderna.

Razón:

- La actualización a la versión `compat` es parte de un proceso de migración recomendado para adaptarse a las nuevas versiones de Angular y Firebase sin romper la compatibilidad existente.

2. Eliminación de `AngularFirestoreDocument` y Simplificación de `getClient`

Cambio:

- Se eliminó el uso de `AngularFirestoreDocument` y `snapshotChanges()` en favor de un enfoque simplificado utilizando `valueChanges()` con un mapeo para manejar el caso donde el documento no existe (`null`).

Razón:

- `valueChanges()` es una forma más sencilla y directa de obtener datos de un documento en Firestore. Maneja automáticamente la suscripción y la transformación de los datos en un observable, simplificando el código y eliminando la necesidad de manejar manualmente la existencia del documento.

Con estos cambios podrás seguir el proyecto sin ningún problema.

¡Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](http://www.globalmentoring.com.mx)